

Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 11 de Mayo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada RESNET

1. Importe de Librerías

```
In [3]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import classification_report

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

```

In [4]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-05-12 07:37:43.658983

2. Definición de Rutas y Parámetros

```

In [6]: data_dir = 'Datos Finales' # Reemplaza con la ruta real
        metadata_file = 'Metadatos_Finales.csv' # Reemplaza con la ruta real
        image_height = 150
        image_width = 150
        batch_size = 15
        num_classes = None # Se determinará automáticamente del archivo CSV

```

3. Cargar y procesar los metadatos

```

In [8]: try:
        metadata = pd.read_csv(metadata_file, sep=";")
        metadata["Planta_Condición_Enfermedad"] = metadata["Planta_Estado_Español"]
        metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})

        # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
        # y 'label' (o el nombre de tu columna de etiquetas) existan.

```

```

if 'image_id' not in metadata.columns or 'Planta_Condicción_Enfermedad' not in metadata.columns:
    raise ValueError("El archivo CSV debe contener las columnas 'image_id' y 'Planta_Condicción_Enfermedad'")
class_names = metadata['Planta_Condicción_Enfermedad'].unique().tolist()
num_classes = len(class_names)
print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file}'")
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()

```

Se encontraron 37 clases: ['Yuca_Enferma_Pudrición de la yuca', 'Yuca_Enferma_Manchas amarillas en la yuca', 'Yuca_Enferma_Venas marrones de la yuca', 'Yuca_Enferma_Manchas verdes en la yuca', 'Yuca_Saludable_Saludable', 'Fresa_Saludable_Saludable', 'Papa_Enferma_Quemado de la papa', 'Maíz_Saludable_Saludable', 'Tomate_Enferma_Pinta en ojo de buey', 'Papa_Enferma_Pudrición de la papa', 'Tomate_Enferma_Pudrición de la papa', 'Tomate_Enferma_Moteado del tomate', 'Tomate_Enferma_Hongo blanco del tomate', 'Manzana_Enferma_Hongo del manzano', 'Tomate_Enferma_Puntos negros del tomate', 'Tomate_Enferma_Quemado de la papa', 'Maíz_Enferma_Moho rojo del maíz', 'Tomate_Saludable_Saludable', 'Tomate_Enferma_Hojas enrolladas del tomate', 'Manzana_Enferma_Costra de la manzana', 'Maíz_Enferma_Quemazón del maíz', 'Tomate_Enferma_Polvillo rojo', 'Tomate_Enferma_Puntitos marrones en las hojas', 'Maíz_Enferma_Pinta gris', 'Manzana_Enferma_Pudrición del fruto', 'Manzana_Saludable_Saludable', 'Fresa_Enferma_Hojas quemadas de la fresa', 'Papa_Saludable_Saludable', 'Coliflor_Enferma_Moho azul', 'Pepino Cohombro_Saludable_Saludable', 'Fríjol_Enferma_Mancha angular', 'Coliflor_Saludable_Saludable', 'Coliflor_Enferma_Pudrición del fruto', 'Fríjol_Enferma_Oxido de frijol', 'Coliflor_Enferma_Podredumbre blanda', 'Fríjol_Saludable_Saludable', 'Pepino Cohombro_Enferma_Enferma']

```

In [9]: train, test = train_test_split(metadata, test_size=0.2, random_state=42, shuffle=True)
        train.shape, test.shape

```

```

Out[9]: ((42354, 13), (10589, 13))

```

4. Crear un generador de datos

```

In [11]: from sklearn.utils.class_weight import compute_class_weight
        import numpy as np

        # Obtener las clases del DataFrame
        labels = train['Planta_Condicción_Enfermedad']
        classes = np.unique(labels)

        # Calcular pesos balanceados
        class_weights = compute_class_weight(class_weight='balanced', classes=classes, y=train['Planta_Condicción_Enfermedad'])
        class_weights_dict = dict(zip(classes, class_weights))

```

```
# Verificar pesos
print("Pesos por clase:", class_weights_dict)
```

```
Pesos por clase: {'Coliflor_Enferma_Moho azu': 8.061286638751428, 'Coliflor_Enferma_Podredumbre blanda': 8.294947121034077, 'Coliflor_Enferma_Pudrición del fruto': 14.308783783783785, 'Coliflor_Saludable_Saludable': 6.937592137592137, 'Fresa_Enferma_Hojas quemadas de la fresa': 1.2905329230019196, 'Fresa_Saludable_Saludable': 3.1361717882265827, 'Fríjol_Enferma_Mancha angular': 3.308389314169661, 'Fríjol_Enferma_Oxido de frijol': 3.279950437543561, 'Fríjol_Saludable_Saludable': 3.3470839260312943, 'Manzana_Enferma_Costra de la manzana': 2.2622583057365664, 'Manzana_Enferma_Hongo del manzano': 5.203194103194103, 'Manzana_Enferma_Pudrición del fruto': 2.3032247539289794, 'Manzana_Saludable_Saludable': 0.8698348804731784, 'Maíz_Enferma_Moho rojo del maíz': 1.1998980112187658, 'Maíz_Enferma_Pinta gris': 2.7919578114700068, 'Maíz_Enferma_Quemazón del maíz': 1.4526684044450542, 'Maíz_Saludable_Saludable': 1.2308631211857017, 'Papa_Enferma_Pudrición de la papa': 1.4308783783783783, 'Papa_Enferma_Quemado de la papa': 1.4308783783783783, 'Papa_Saludable_Saludable': 9.382809038546743, 'Pepino Cohombro_Enferma_Enferma': 4.088223938223938, 'Pepino Cohombro_Saludable_Saludable': 4.193050193050193, 'Tomate_Enferma_Hojas enrolladas del tomate': 0.26714182093411964, 'Tomate_Enferma_Hongo blanco del tomate': 1.5022345179825496, 'Tomate_Enferma_Moteado del tomate': 3.8412842372573914, 'Tomate_Enferma_Pinta en ojo de buey': 1.0193256479988448, 'Tomate_Enferma_Polvillo rojo': 0.8536187193905315, 'Tomate_Enferma_Pudrición de la papa': 0.7496415865767536, 'Tomate_Enferma_Puntitos marrones en las hojas': 0.8078353582940739, 'Tomate_Enferma_Puntos negros del tomate': 0.672958672958673, 'Tomate_Enferma_Quemado de la papa': 1.4308783783783783, 'Tomate_Saludable_Saludable': 0.8992165771427358, 'Yuca_Enferma_Manchas amarillas en la yuca': 0.10875001925733448, 'Yuca_Enferma_Manchas verdes en la yuca': 0.5996347316410177, 'Yuca_Enferma_Pudrición de la yuca': 1.3157502329916122, 'Yuca_Enferma_Venas marrones de la yuca': 0.6537422631083396, 'Yuca_Saludable_Saludable': 0.5554113065030095}
```

```
In [12]: datagen = ImageDataGenerator(
    validation_split = 0.2,
    preprocessing_function = None,
    rotation_range = 45,
    zoom_range = 0.2,
    horizontal_flip = True,
    vertical_flip = True,
    fill_mode = 'nearest',
    shear_range = 0.1,
    height_shift_range = 0.1,
    width_shift_range = 0.1
)

train_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
```

```

y_col='Planta Condición Enfermedad',
target_size=(image_height, image_width),
batch_size=batch_size,
class_mode='categorical', # Importante para clasificación multiclase
subset='training',
seed=42,
classes=class_names # Asegurarse del orden de las clases
)

validation_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta Condición Enfermedad',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=42,
    classes=class_names
)

```

Found 33881 validated image filenames belonging to 37 classes.

Found 8470 validated image filenames belonging to 37 classes.

5. Cargar el modelo ResNet50 preentrenado (sin la capa superior de clasificación)

```
In [14]: base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(in
```

6. Congelar las capas convolucionales base

```
In [16]: for layer in base_model.layers:
        layer.trainable = False
```

7. Añadir una nueva capa de clasificación personalizada

```
In [18]: x = base_model.output
x = GlobalAveragePooling2D()(x)
# Puedes añadir más capas densas aquí si lo deseas
# x = Dense(1024, activation='relu')(x)
predictions = Dense(num_classes, activation='softmax')(x)
```

8. Crear el modelo final

```
In [20]: model = Model(inputs=base_model.input, outputs=predictions)
```

9. Compilar el modelo para fine-tuning de la última capa

```
In [22]: model.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])
```

10. Entrenar el modelo con la última capa descongelada

```
In [24]: epochs_fine_tune = 20 # Define el número de épocas para el fine-tuning
```

```
In [25]: print("Comenzando el entrenamiento de la última capa...")
history_fine_tune = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=epochs_fine_tune,
    verbose = 1
)
```

Comenzando el entrenamiento de la última capa...

Epoch 1/20

2258/2258 ————— 748s 330ms/step - accuracy: 0.6585 - loss: 1.1622 - val_accuracy: 0.7628 - val_loss: 0.7015

Epoch 2/20

2258/2258 ————— 154s 68ms/step - accuracy: 0.7333 - loss: 0.6186 - val_accuracy: 0.7641 - val_loss: 0.6947

Epoch 3/20

2258/2258 ————— 781s 346ms/step - accuracy: 0.7809 - loss: 0.6747 - val_accuracy: 0.7832 - val_loss: 0.6865

Epoch 4/20

2258/2258 ————— 156s 69ms/step - accuracy: 1.0000 - loss: 0.1573 - val_accuracy: 0.7832 - val_loss: 0.7044

Epoch 5/20

2258/2258 ————— 793s 351ms/step - accuracy: 0.7931 - loss: 0.6352 - val_accuracy: 0.8073 - val_loss: 0.6193

Epoch 6/20

2258/2258 ————— 159s 70ms/step - accuracy: 0.9333 - loss: 0.4164 - val_accuracy: 0.8007 - val_loss: 0.6261

Epoch 7/20
2258/2258 ————— **831s** 368ms/step – accuracy: 0.8039 – loss: 0.6049 – val_accuracy: 0.8018 – val_loss: 0.6561
 Epoch 8/20
2258/2258 ————— **199s** 88ms/step – accuracy: 0.8667 – loss: 0.3619 – val_accuracy: 0.7979 – val_loss: 0.6961
 Epoch 9/20
2258/2258 ————— **11179s** 5s/step – accuracy: 0.8094 – loss: 0.5952 – val_accuracy: 0.7862 – val_loss: 0.6992
 Epoch 10/20
2258/2258 ————— **157s** 69ms/step – accuracy: 0.8000 – loss: 0.6929 – val_accuracy: 0.7934 – val_loss: 0.6666
 Epoch 11/20
2258/2258 ————— **829s** 367ms/step – accuracy: 0.8106 – loss: 0.5807 – val_accuracy: 0.8073 – val_loss: 0.6164
 Epoch 12/20
2258/2258 ————— **171s** 76ms/step – accuracy: 1.0000 – loss: 0.0964 – val_accuracy: 0.8156 – val_loss: 0.5872
 Epoch 13/20
2258/2258 ————— **941s** 417ms/step – accuracy: 0.8234 – loss: 0.5555 – val_accuracy: 0.7637 – val_loss: 0.7486
 Epoch 14/20
2258/2258 ————— **197s** 87ms/step – accuracy: 0.8000 – loss: 0.5605 – val_accuracy: 0.7641 – val_loss: 0.7233
 Epoch 15/20
2258/2258 ————— **997s** 441ms/step – accuracy: 0.8235 – loss: 0.5577 – val_accuracy: 0.8052 – val_loss: 0.6092
 Epoch 16/20
2258/2258 ————— **193s** 85ms/step – accuracy: 0.9333 – loss: 0.3024 – val_accuracy: 0.8117 – val_loss: 0.5942
 Epoch 17/20
2258/2258 ————— **939s** 416ms/step – accuracy: 0.8203 – loss: 0.5508 – val_accuracy: 0.8186 – val_loss: 0.6216
 Epoch 18/20
2258/2258 ————— **184s** 82ms/step – accuracy: 0.8667 – loss: 0.1915 – val_accuracy: 0.8215 – val_loss: 0.6273
 Epoch 19/20
2258/2258 ————— **895s** 396ms/step – accuracy: 0.8278 – loss: 0.5521 – val_accuracy: 0.8132 – val_loss: 0.6093
 Epoch 20/20
2258/2258 ————— **177s** 78ms/step – accuracy: 0.9333 – loss: 0.2461 – val_accuracy: 0.8136 – val_loss: 0.6349

```
In [26]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-05-12 13:22:24.360200

Duración: 5:44:40.701217

11. Opcional: Descongelar algunas de las últimas capas del modelo base para un fine-tuning más profundo

```
In [28]: unfreeze_layers = 2 # Número de capas a descongelar (contando desde la última)

for layer in base_model.layers[-unfreeze_layers:]:
    layer.trainable = True
```

12. Opcional: Recompilar el modelo después de descongelar las capas

```
In [30]: model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # Usa Adam
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])
```

13. Opcional: Entrenar el modelo con las capas descongeladas

```
In [32]: epochs_deep_fine_tune = 5 # Define el número de épocas para el fine-tuning

print(f"\nComenzando el fine-tuning profundo de las últimas {unfreeze_layers} capas")
history_deep_fine_tune = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=epochs_deep_fine_tune
)
```


Comenzando el fine-tuning profundo de las últimas 2 capas...

Epoch 1/5

2258/2258 ————— **873s** 385ms/step – accuracy: 0.8466 – loss: 0.4625 – val_accuracy: 0.8293 – val_loss: 0.5296

Epoch 2/5

2258/2258 ————— **170s** 75ms/step – accuracy: 0.6667 – loss: 0.8802 – val_accuracy: 0.8292 – val_loss: 0.5458

Epoch 3/5

2258/2258 ————— **860s** 381ms/step – accuracy: 0.8514 – loss: 0.4261 – val_accuracy: 0.8322 – val_loss: 0.5239

Epoch 4/5

2258/2258 ————— **184s** 81ms/step – accuracy: 0.8000 – loss: 0.5986 – val_accuracy: 0.8298 – val_loss: 0.5209

Epoch 5/5

2258/2258 ————— **886s** 392ms/step – accuracy: 0.8562 – loss: 0.4153 – val_accuracy: 0.8359 – val_loss: 0.5096

14. Evaluar el modelo final

```
In [34]: print("\nEvaluando el modelo...")
evaluation_result = model.evaluate(validation_generator)
print(f"Pérdida en la validación: {evaluation_result[0]:.4f}")
print(f"Precisión en la validación: {evaluation_result[1]:.4f}")
```

Evaluando el modelo...

565/565 ————— **171s** 303ms/step – accuracy: 0.8400 – loss: 0.5174

Pérdida en la validación: 0.5223

Precisión en la validación: 0.8349

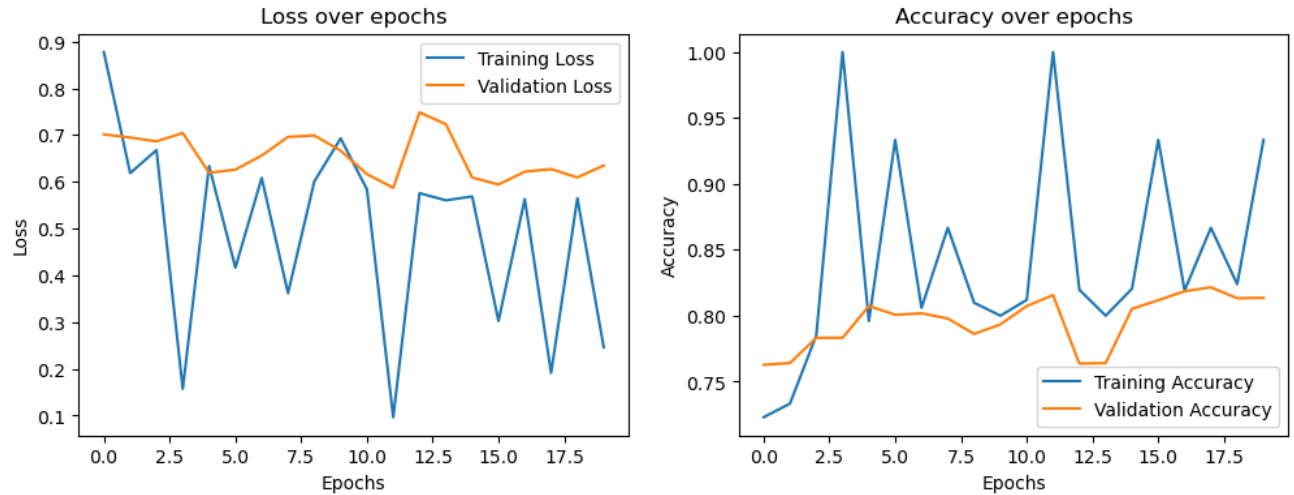
```
In [35]: # Acceder a los datos de pérdida y precisión
loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
accuracy = history_fine_tune.history['accuracy']
val_accuracy = history_fine_tune.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_fine_tune.history['loss'], label='Training Loss')
plt.plot(history_fine_tune.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history_fine_tune.history['accuracy'], label='Training Accuracy')
```

```
plt.plot(history_fine_tune.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

Out[35]: <matplotlib.legend.Legend at 0x2a2f310a0>



```
In [36]: epochs = list(range(1, 21))
data = {
    'Epoch': epochs,
    'Training Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	0.877623	0.701454	0.722967
1	2	0.618563	0.694715	0.733333
2	3	0.667885	0.686530	0.785006
3	4	0.157317	0.704405	1.000000
4	5	0.633890	0.619341	0.795931
5	6	0.416386	0.626051	0.933333
6	7	0.608463	0.656111	0.805971
7	8	0.361866	0.696136	0.866667
8	9	0.600852	0.699174	0.809780
9	10	0.692859	0.666646	0.800000
10	11	0.583945	0.616365	0.811965
11	12	0.096436	0.587200	1.000000
12	13	0.575720	0.748572	0.819672
13	14	0.560472	0.723277	0.800000
14	15	0.568649	0.609200	0.820439
15	16	0.302357	0.594192	0.933333
16	17	0.563203	0.621597	0.819081
17	18	0.191473	0.627279	0.866667
18	19	0.565063	0.609317	0.823924
19	20	0.246079	0.634857	0.933333

	Validation Accuracy
0	0.762766
1	0.764066
2	0.783215
3	0.783215
4	0.807329
5	0.800709
6	0.801773
7	0.797872
8	0.786170
9	0.793381
10	0.807329
11	0.815603
12	0.763712
13	0.764066
14	0.805201
15	0.811702
16	0.818558
17	0.821513
18	0.813239
19	0.813593

```
In [37]: # Accuracy final en entrenamiento
val_loss = history_fine_tune.history['val_loss'][-1]
```

```
In [38]: print('Perdida final en Train:', history_fine_tune.history['loss'][-1])
print("Pérdida final en Validation:", history_fine_tune.history['val_loss'][-1])
```

```
print("Accuracy final en Train:", history_fine_tune.history['accuracy'][-1])
print("Accuracy final en Validation:", history_fine_tune.history['val_accuracy'][-1])
```

Perdida final en Train: 0.24607905745506287
 Pérdida final en Validation: 0.6348571181297302
 Accuracy final en Train: 0.9333333373069763
 Accuracy final en Validation: 0.8135933876037598

```
In [39]: class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo14_copy.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Yuca_Enferma_Pudrición de la yuca', 1: 'Yuca_Enferma_Manchas amarillas
en la yuca', 2: 'Yuca_Enferma_Venas marrones de la yuca', 3: 'Yuca_Enferma_M
anchas verdes en la yuca', 4: 'Yuca_Saludable_Saludable', 5: 'Fresa_Saludabl
e_Saludable', 6: 'Papa_Enferma_Quemado de la papa', 7: 'Maíz_Saludable_Salud
able', 8: 'Tomate_Enferma_Pinta en ojo de buey', 9: 'Papa_Enferma_Pudrición
de la papa', 10: 'Tomate_Enferma_Pudrición de la papa', 11: 'Tomate_Enferma_
Moteado del tomate', 12: 'Tomate_Enferma_Hongo blanco del tomate', 13: 'Manz
ana_Enferma_Hongo del manzano', 14: 'Tomate_Enferma_Puntos negros del tomat
e', 15: 'Tomate_Enferma_Quemado de la papa', 16: 'Maíz_Enferma_Moho rojo del
maíz', 17: 'Tomate_Saludable_Saludable', 18: 'Tomate_Enferma_Hojas enrollada
s del tomate', 19: 'Manzana_Enferma_Costra de la manzana', 20: 'Maíz_Enferma
_Quemazón del maíz', 21: 'Tomate_Enferma_Polvillo rojo', 22: 'Tomate_Enferma
_Puntitos marrones en las hojas', 23: 'Maíz_Enferma_Pinta gris', 24: 'Manzan
a_Enferma_Pudrición del fruto', 25: 'Manzana_Saludable_Saludable', 26: 'Fres
a_Enferma_Hojas quemadas de la fresa', 27: 'Papa_Saludable_Saludable', 28: '
Coliflor_Enferma_Moho azu', 29: 'Pepino Cohombro_Saludable_Saludable', 30: '
Fríjol_Enferma_Mancha angular', 31: 'Coliflor_Saludable_Saludable', 32: 'Col
iflor_Enferma_Pudrición del fruto', 33: 'Fríjol_Enferma_Oxido de frijol', 3
4: 'Coliflor_Enferma_Podredumbre blanda', 35: 'Fríjol_Saludable_Saludable',
36: 'Pepino Cohombro_Enferma_Enferma'}
```

15. Testeo del Modelo

```
In [43]: test.Planta_Condición_Enfermedad = test.Planta_Condición_Enfermedad.astype('float32')

datagen = ImageDataGenerator(
    validation_split = 0.2,
    preprocessing_function = None,
    rotation_range = 45,
    zoom_range = 0.2,
    horizontal_flip = True,
    vertical_flip = True,
    fill_mode = 'nearest',
    shear_range = 0.1,
```

```

height_shift_range = 0.1,
width_shift_range = 0.1)

test_generator = datagen.flow_from_dataframe(test,
                                             directory = "Datos Finales",
                                             subset = "training",
                                             x_col = "image_id",
                                             y_col = "Planta_Condición_Enfermedad",
                                             target_size = (image_height,image_width),
                                             batch_size = batch_size,
                                             class_mode = "categorical")

```

Found 8471 validated image filenames belonging to 37 classes.

```

In [44]: # -----
# 1. Obtener predicciones completas para test
# -----
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes

```

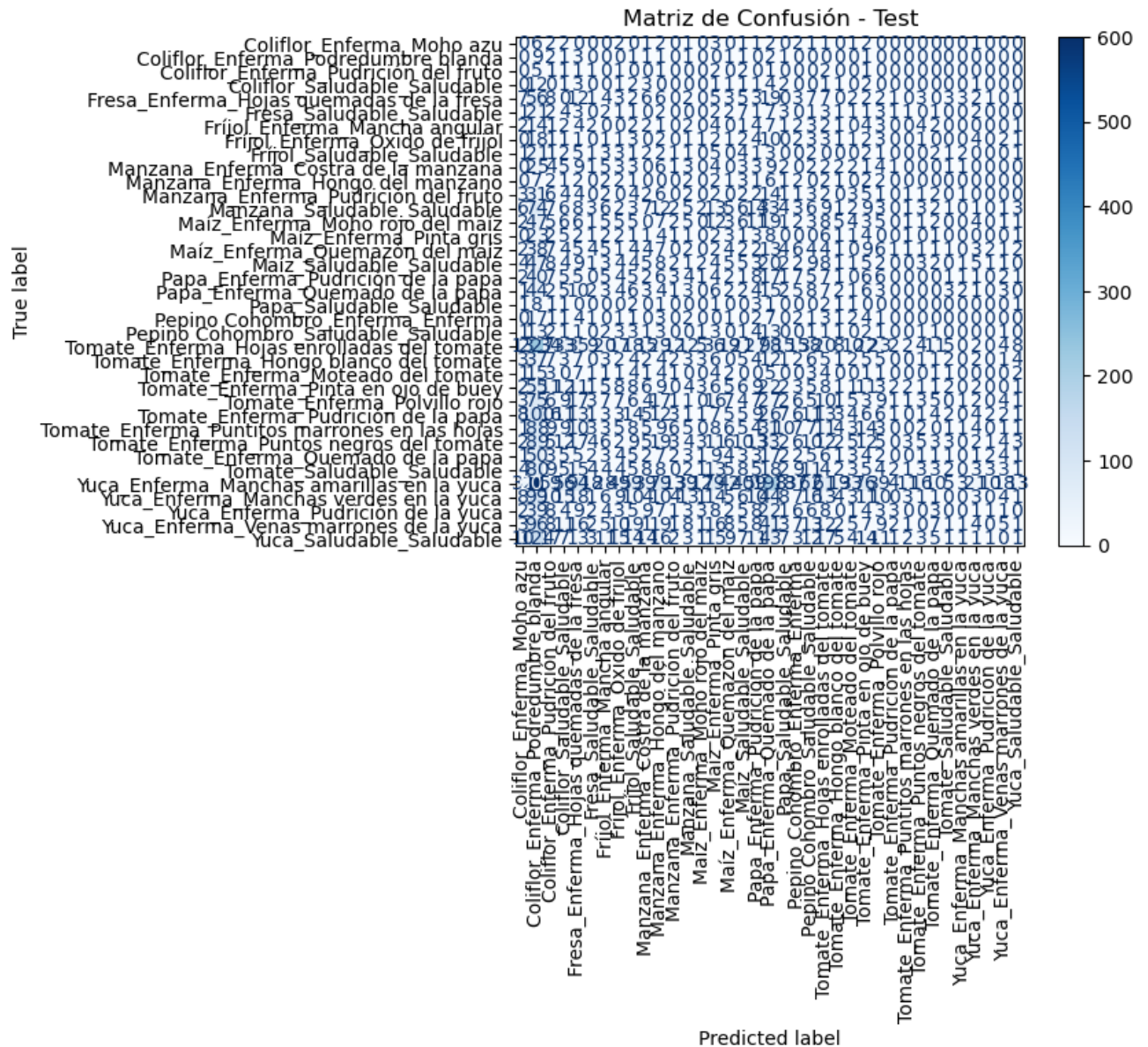
565/565 ————— 142s 251ms/step

```

In [46]: # -----
# 2. Matriz de Confusión
# -----
plt.figure(figsize=(200, 80))
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener
disp.plot(cmap="Blues", xticks_rotation=90)
plt.title("Matriz de Confusión - Test")
plt.savefig("matriz_confusion_test_modelo_11.png", dpi=300, bbox_inches='tight')
plt.show()

```

<Figure size 20000x8000 with 0 Axes>



```
In [47]: accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.0153

```
In [48]: # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())
```

```
# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=1.05)
plt.show()
```

1/1 ————— 1s 760ms/step

```

-----
ValueError                                Traceback (most recent call last)
Cell In[48], line 28
    26 plt.figure(figsize=(15, 8))
    27 for i in range(num_samples):
--> 28     plt.subplot(3, 4, i + 1)
    29     img_to_show = normalize_for_display(x_batch[i])
    30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs)))
    ):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13

```




```
In [55]: # 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

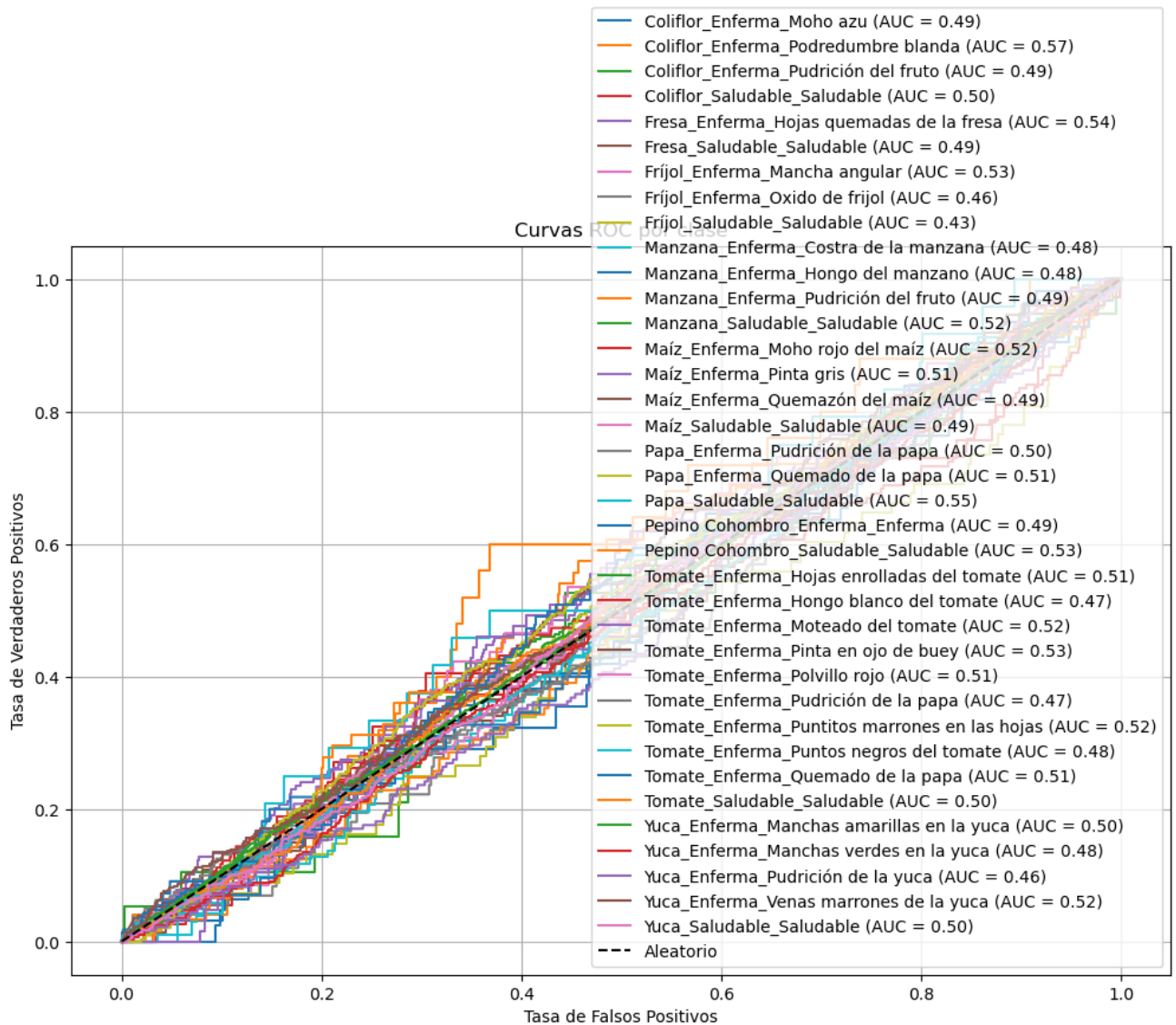
# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



```
In [57]: from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

AUC Macro promedio: 0.5013

AUC Micro promedio: 0.4569

```
In [59]: # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes
```

```

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))

```

Accuracy: 0.0153
 F1 Score (macro): 0.0134
 F1 Score (micro): 0.0153
 F1 Score (weighted): 0.0152

Reporte de clasificación por clase:

		precision	recall	f1-score
support				
	Coliflor_Enferma_Moho azu	0.00	0.00	0.00
31	Coliflor_Enferma_Podredumbre blanda	0.00	0.36	0.01
25	Coliflor_Enferma_Pudrición del fruto	0.00	0.05	0.01
19	Coliflor_Saludable_Saludable	0.00	0.03	0.01
37	Fresa_Enferma_Hojas quemadas de la fresa	0.03	0.07	0.04
179	Fresa_Saludable_Saludable	0.00	0.00	0.00
74	Fríjol_Enferma_Mancha angular	0.00	0.00	0.00
71	Fríjol_Enferma_Oxido de frijol	0.01	0.01	0.01
72	Fríjol_Saludable_Saludable	0.00	0.01	0.01
68	Manzana_Enferma_Costra de la manzana	0.00	0.00	0.00
102	Manzana_Enferma_Hongo del manzano	0.00	0.00	0.00
42	Manzana_Enferma_Pudrición del fruto	0.00	0.00	0.00

105	Manzana_Saludable_Saludable	0.04	0.02	0.02
263	Maíz_Enferma_Moho rojo del maíz	0.00	0.00	0.00
188	Maíz_Enferma_Pinta gris	0.01	0.03	0.01
77	Maíz_Enferma_Quemazón del maíz	0.03	0.03	0.03
153	Maíz_Saludable_Saludable	0.03	0.03	0.03
182	Papa_Enferma_Pudrición de la papa	0.03	0.05	0.04
160	Papa_Enferma_Quemado de la papa	0.02	0.09	0.03
160	Papa_Saludable_Saludable	0.00	0.00	0.00
24	Pepino Cohombro_Enferma_Enferma	0.01	0.04	0.02
55	Pepino Cohombro_Saludable_Saludable	0.00	0.02	0.01
61	Tomate_Enferma_Hojas enrolladas del tomate	0.07	0.02	0.03
869	Tomate_Enferma_Hongo blanco del tomate	0.01	0.01	0.01
145	Tomate_Enferma_Moteado del tomate	0.00	0.00	0.00
63	Tomate_Enferma_Pinta en ojo de buey	0.04	0.05	0.04
228	Tomate_Enferma_Polvillo rojo	0.05	0.03	0.04
276	Tomate_Enferma_Pudrición de la papa	0.05	0.00	0.01
316	Tomate_Enferma_Puntitos marrones en las hojas	0.00	0.00	0.00
284	Tomate_Enferma_Puntos negros del tomate	0.08	0.02	0.03
331	Tomate_Enferma_Quemado de la papa	0.01	0.01	0.01
165	Tomate_Saludable_Saludable	0.05	0.01	0.01
259	Yuca_Enferma_Manchas amarillas en la yuca	0.17	0.00	0.00
2051	Yuca_Enferma_Manchas verdes en la yuca	0.04	0.01	0.01
372	Yuca_Enferma_Pudrición de la yuca	0.04	0.01	0.01
182	Yuca_Enferma_Venas marrones de la yuca	0.08	0.01	0.02
358	Yuca_Saludable_Saludable	0.02	0.00	0.00

424

	accuracy			0.02
8471				
	macro avg	0.03	0.03	0.01
8471				
	weighted avg	0.07	0.02	0.02
8471				

16. Guardar el modelo entrenado

```
In [62]: model.save('modelo_14_resnet50_fine_tuned_class_weighted_unfreeze.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
In [64]: class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
```

```
In [ ]:
```